

Criação de prompts, geração automática de consultas e validação de resultados

DataChat NoSQL — sistema de consulta em linguagem natural sobre MongoDB, com tradução de perguntas por meio de um modelo de linguagem. Esta apresentação abrange os itens do cronograma do projeto previstos para a Semana 2.

Gabriel Azevedo Lira de Farias · João Pedro de Queiroz Dantas · Vitor Jesus Mamede Soares · Vítor Raimundo Fernandes Gabínio

Escopo definido pelo cronograma do projeto

O item "desenvolver consultas MongoDB" foi entregue na Semana 1, na forma de oito consultas documentadas. Os itens restantes, cobertos nesta apresentação, são os seguintes.

ITEM 1 Criação de prompts core/esquema.py · core/tradutor.py	ITEM 2 Geração automática de consultas core/tradutor.py · RF04	ITEM 3 Validação de resultados scripts/comparar_llms.py
---	---	--

A integração completa do backend, o histórico de consultas e o tratamento de erros correspondem à Semana 3 no cronograma do projeto; a interface corresponde à Semana 4. Essa divisão é detalhada no Slide 8.

Estrutura do prompt de sistema

O prompt utilizado é composto por três elementos: o contexto de esquema da base de dados, o formato de saída exigido do modelo e um conjunto de exemplos de referência (few-shot).

Exemplo 1

Filtro simples — estágios **\$match**, **\$sort** e **\$limit**, sem uso de `find()` separado

Exemplo 2

Junção entre coleções — **\$lookup** e **\$unwind**, identificado como o padrão de maior dificuldade de geração

Exemplo 3

Esquema aberto — **\$objectToArray** e **\$unwind**, para descoberta dinâmica de campos em details

```
// core/esquema.py — o contexto não é uma string fixa
price: double | null (33,4% dos documentos) — atenção ao filtrar
store: string (98,5% dos documentos)
```


Comparação empírica entre modelos

Antes da definição do prompt, foram avaliadas as mesmas sete perguntas de maior complexidade do projeto em dois modelos gratuitos, sob as mesmas condições. Ver `scripts/comparar_llms.py`.

	GEMINI · GEMINI-FLASH-LITE-LATEST	LLAMA 3.2 3B · EXECUÇÃO LOCAL
Respostas com JSON válido	7 de 7	4 de 7
Execução sem erro no MongoDB	7 de 7	1 de 7
Tempo médio de resposta	4,6 segundos	7,3 segundos
Custo	Gratuito	Gratuito

Falhas observadas no modelo local

O ambiente de desenvolvimento é um MacBook Air M1 com 8 GB de memória RAM unificada, o que inviabilizou o uso do Llama 3 8B (aproximadamente 4,7 GB apenas para o modelo). Foi avaliado o maior modelo compatível: Llama 3.2 3B (aproximadamente 2 GB).

JSON inválido Omissão de aspas em operadores como **\$gt** e **\$avg**; interrupção da resposta no meio de um pipeline extenso

Location 28811 Uso de uma opção inexistente no MongoDB — **preserveNullValues** dentro do estágio **\$unwind**

Location 17276 Referência a uma variável indefinida dentro do operador **\$objectToArray** — **\$\$value** não é válida nesse contexto

Implementação da tradução pergunta–consulta

Implementado em `core/tradutor.py`: uma única função realiza a chamada ao modelo Gemini e retorna a saída em um formato estruturado padronizado.

```
traduzir(pergunta, esquema_contexto, cliente)

// parâmetros da chamada
temperature = 0
response_mime_type = "application/json"

// formato de saída padronizado
{ "colecacao": "reviews" | "products", "pipeline": [...] }
```


Critérios de validação

Este processo não deve ser confundido com o validador de segurança, previsto para a Semana 3.

Nesta etapa, validar significa confirmar que o pipeline gerado é executável e produz um resultado coerente com a pergunta original.

JSON VÁLIDO

7 de 7

Todas as respostas do modelo foram interpretadas sem erro

EXECUÇÃO SEM ERRO NO MONGODB

7 de 7

Consulta agregada real, executada contra a base carregada

Itens não incluídos nesta entrega

Um executor mínimo já existe no repositório, necessário apenas para a execução das consultas na etapa de validação (Slide 7). Os itens a seguir, no entanto, não são apresentados como entrega da Semana 2.

ITEM	PROTÓTIPO EM	PREVISTO PARA
Integração completa entre MongoDB e o LLM	core/orquestrador.py	Semana 3
Bloqueio de operações destrutivas (RF07)	core/validador.py	Semana 3
Histórico de consultas (RF08)	app/main.py	Semana 3
Tratamento de erros (RF09)	core/orquestrador.py	Semana 3
Interface final integrada	app/main.py	Semana 4

Concluídas as etapas de criação de prompts e geração automática de consultas, a próxima fase, prevista para a Semana 3, é a integração completa do backend.

- **Integração entre MongoDB e o LLM** em um fluxo único de backend, substituindo os scripts isolados atuais
- **Formalização do bloqueio de operações destrutivas** (RF07), acompanhada de testes adversariais
- **Implementação do histórico de consultas** (RF08)
- **Tratamento de erros** — pergunta ambígua, campo inexistente e consulta sem resultado (RF09)